Amaura Labs - Stage 2

Execution-Based Evaluation and Selection vs. Execution-Blind Frontier-LLM Judgment Under a Matched Candidate-Generation Budget

Extremely Detailed Public-Release Research Manuscript | Release 1.1 (Densified and Expanded)

Amaura Labs Research | 7 October 2026 | State: REPORT_COMPLETE

Primary frontier model: NVIDIA Nemotron 3 Super 120B (nvidia/nemotron-3-super-120b-a12b)

Public-release edition: consolidated methodology, complete trial results, statistical analysis, security and audit record, failure analysis, reproducibility notes, and claim boundaries.

REPORT_COMPLETE | 7 October 2026 | Amaura Labs Research | Extremely Detailed Public Release

Public-release status

The strongest defensible claim is that empirical execution provides a better selection signal than execution-blind frontier-model judgment under this matched-pool protocol. The result is not evidence of global optimality, universal superiority, or an autonomous scientist.

Interpretation

Study	Graph Coloring + PFSP	**Trials**	60 total (30 + 30)
Candidate generation	2,400 calls	**Blind judge**	180 calls / 3 shuffles
Hidden benchmark	200 GC + 204 PFSP	**Primary model**	Nemotron 3 Super 120B
GC primary	D: 100% valid; B: 30% valid	**GC D vs B**	27 - 3; p = 3.83e-6
PFSP primary	D: 96.7% valid; B: 66.7% valid	**PFSP D vs B**	-822.9 valid-only; p = 0.0086

YES - under the Stage 2 protocol, execution-based selection (Arm D) outperformed execution-blind judgment (Arm B) on the primary comparisons and sharply reduced hidden runtime failures.

Primary result

At a matched candidate-generation budget, does selecting algorithmic candidates by actually executing them on development instances add measurable value over a strong execution-blind frontier-LLM judge?

Central question

STAGE 2 - PUBLIC-RELEASE RESEARCH HIGHLIGHTS

Abstract

Amaura Labs Stage 2 evaluates a narrow but important question in AI-assisted algorithm discovery: when a language model generates a fixed pool of candidate algorithms, does selecting candidates by actually executing and measuring them add measurable value over selecting from the same pool using a strong execution-blind language-model judge? The study was designed to isolate selection rather than candidate generation by holding the expert candidate pool constant for the primary Arm B versus Arm D comparison. Two combinatorial optimization domains were used: Graph Coloring and Permutation Flow Shop Scheduling (PFSP). The study completed 60 paired trials, 30 per domain, with 20 candidates per primary comparison pool and 40 candidate-generation calls per trial across the complete A/B/C/D design, yielding 2,400 candidate-generation calls and 180 blind-judge calls. Hidden evaluation covered 200 unseen Graph Coloring instances and 204 unseen PFSP instances.

The execution-selected expert arm (Arm D) achieved 100.0% hidden validity on Graph Coloring and 96.7% on PFSP. The execution-blind judge (Arm B) failed on 70.0% of Graph Coloring trials and 33.3% of PFSP trials. On Graph Coloring, Arm D won 27 of 30 paired trials against Arm B; on the subset where both systems produced valid hidden results, Arm D averaged 1,227.0 colors versus 1,240.22 for Arm B. On PFSP, among the 19 trials where both arms were valid, Arm D averaged 1,072,414.05 makespan versus 1,073,236.95 for Arm B, a paired difference of -822.90 and Wilcoxon $p = 0.0086$. The experiment also reports selection regret, classical baselines, failure modes, cryptographic identity checks, hidden-data firewalls, model-provenance checks, benchmark split disjointness checks, and a dedicated audit of a timeout-handling defect discovered during the run.

The strongest interpretation is intentionally narrower than a claim of general autonomous scientific discovery. Stage 2 supports the statement that, on these preregistered benchmark families and at this matched candidate-generation budget, empirical execution is a materially better selection signal than execution-blind frontier-LLM judgment. It does not establish global optimality, universal superiority over all search methods, superiority to state-of-the-art combinatorial solvers, or autonomous scientist-level capability. A further methodological caveat is material for PFSP: the preregistered crash penalty of 999,999 is numerically below several valid aggregate makespans, so raw scalar penalty aggregation is not semantically valid for PFSP; the manuscript therefore emphasizes validity-aware analyses and separately reports the penalty-based distribution as an audit artifact.

Document map

Section	Purpose
1. Research context	What Stage 2 was designed to establish and what it deliberately did not attempt.
2. Central question and hypotheses	Precise estimand, primary contrast, and interpretation rules.
3. Related work	How Stage 2 relates to FunSearch, AlphaEvolve, and prior LLM program synthesis.
4. Experimental design	Arms A-D, matched candidate pools, budgets, randomization, and trial structure.
5. Benchmark and objective construction	Graph Coloring and PFSP problem definitions and hidden evaluation structure.
6. Evaluator and security architecture	Sandboxing, hidden firewalls, AST checks, timeout control, and correctness checks.
7. Audit and remediation chronology	Adversarial findings, benchmark leakage, provenance, security, and timeout defect.
8. Complete numerical results	At-a-glance results and full trial tables.
9. Statistical analysis	Wilcoxon tests, bootstrap intervals, effect sizes, Holm correction, regret.
10. Failure analysis	What failed, where it failed, and what those failures imply.
11. Classical baselines and mechanism	Comparison against established heuristics and algorithmic interpretation.
12. Compute, token, and provenance accounting	Inference budget and fairness checks.
13. Reproducibility and artifact release	What a reviewer should inspect to reproduce the result.
14. Limitations and threats to validity	Important ways the result can be overstated or fail to generalize.
15. Conclusion	The strongest defensible Stage 2 claim.
Appendices	Equations, audit checklist, data dictionary, trial-level audit notes, references.

1. Research context and motivation

1.1 The underlying Amaura thesis

Amaura Labs is built around a simple systems abstraction: define an objective and constraints; generate candidate programs or strategies; execute them safely; measure them on a development evaluator; reject invalid or poor candidates; and preserve the best verified candidates. The key scientific question is not whether language models can emit code. Existing work already establishes that. The question is whether an objective evaluator can turn noisy generation into a more reliable search process.

Stage 2 intentionally narrows that broader thesis. It does not study long-horizon evolution, unlimited generations, global optimization, or open-ended science. Instead, it asks whether an LLM can make a better choice when it is allowed to see the consequences of executing candidates, compared with a strong LLM judge that must choose without running those candidates. This isolates the value of empirical execution as a selection signal.

1.2 Why the isolation matters

A common experimental confound in algorithm-discovery systems is that the generation prompt, search strategy, selection rule, evaluator, and candidate pool all change simultaneously. An observed improvement then cannot be attributed cleanly to any one component. Stage 2 therefore made the primary Arm B versus Arm D comparison unusually strict: both arms operate on the same expert-generated 20-candidate pool. The only intended difference is the selection mechanism. Arm B selects with an execution-blind language-model judge; Arm D evaluates candidates on a development set and selects empirically.

The wider A/B/C/D design additionally tests random selection and a diversified candidate pool, but these arms are secondary to the primary B-versus-D contrast. This distinction is central to interpreting the result.

1.3 Scope of the result

- Positive evidence for empirical execution as a selection signal in two combinatorial algorithmic domains.
- Evidence that execution can sharply reduce hidden runtime failure when candidate code is noisy.
- Evidence that the effect is not solely a crash phenomenon: on dual-valid Graph Coloring trials, Arm D still had the lower mean score and a confidence interval excluding zero; on valid PFSP trials, Arm D significantly reduced makespan.
- No proof of global optimality, no proof that evolution is superior to independent search, and no proof of universal dominance over classical metaheuristics.
- No commercial validation. Stage 2 is a scientific systems experiment, not product-market-fit evidence.

2. Central question, hypotheses, and interpretation rules

Primary research question: "At a matched candidate-generation budget, does choosing algorithmic candidates by actually executing them on development instances add measurable value over a strong prompt that chooses without running anything?"

2.1 Primary contrast

The primary contrast is Arm D versus Arm B. Both receive the same 20 expert candidates per trial. Arm B sees the candidates without execution results and uses a frontier-model judge. Arm D executes the same candidates on development instances and selects the best valid candidate by measured objective score. Thus the candidate pool is held constant while the information available to the selector changes.

2.2 Hypothesis

H1: Execution-based selection provides a lower hidden objective than execution-blind frontier-LLM judgment under the same candidate-generation budget. Because both Graph Coloring and PFSP are minimization problems, a negative D - B difference favors Arm D.

2.3 What counts as success

- Hidden performance is evaluated only after selection is frozen; hidden data are not available to the selector.
- A valid executable candidate must satisfy the domain-specific output constraints and complete under the evaluator timeout.
- Study-level conclusions rely on paired trial units, not treating each hidden instance as an independent replicate.

- Where the raw crash penalty has semantic problems, validity-aware and dual-valid analyses are reported rather than allowing a misleading scalar score to dominate interpretation.
- Claims are limited to the actual benchmark families, candidate-generation budget, model configuration, and evaluation protocol used.

3. Related work and novelty positioning

Stage 2 is explicitly not claiming that Amaura invented evaluator-guided LLM search. The broad paradigm is well established. FunSearch pairs a pretrained LLM with a systematic evaluator and uses iterative program search to discover improved constructions and algorithms. AlphaEvolve extends the idea to an evolutionary coding agent that proposes programs, evaluates them, and feeds promising programs back into search. Earlier program-synthesis work also documents the growing use of LLMs for code generation and the relationship between model scale and synthesis performance. [1-4]

3.1 What Stage 2 adds experimentally

The narrower experimental contribution is a controlled comparison of selection mechanisms under matched candidate pools. Rather than arguing from a system demonstration that execution "helps," Stage 2 asks whether an execution-blind frontier judge can substitute for actual execution when choosing among the same generated candidates. In this setup, the execution signal is a direct measurement from the evaluator rather than an inferred quality judgment.

3.2 Distinction from open-ended evolution

FunSearch and AlphaEvolve address broader evolutionary loops in which programs can generate descendants and the system uses evaluated fitness to shape future search. Stage 2 deliberately does not test that full claim. Its candidate budget is bounded; the primary pool is fixed; and the core contrast is selection with versus without execution. This makes the causal interpretation narrower and more defensible.

3.3 Classical optimization context

The benchmark domains also have substantial pre-LLM literature. For Graph Coloring, DSATUR, Recursive Largest First (RLF), and tabu-search methods such as TabuCol provide recognizable reference points. For PFSP, the NEH constructive heuristic is a classic benchmark, while Iterated Greedy is a widely used metaheuristic family. [5-9] Stage 2 uses these methods as contextual baselines, not as a claim that the LLM system has solved either problem optimally.

4. Experimental design

4.1 Four-arm structure

Arm	Candidate generation	Selection	Primary role
A - Random	20 expert candidates	Seeded random draw	No-signal selection control
B - Blind Judge	20 expert candidates	Execution-blind frontier LLM judge	Primary comparator
C - Diversified	20 domain-specific perspectives	Execute on development set	Diversity + execution arm
D - Expert	20 expert candidates	Execute on development set	Primary execution-selected arm

The complete study comprises 30 trials for Graph Coloring and 30 trials for PFSP. Each trial contributes 20 expert candidates and 20 diversified candidates to the wider four-arm design, yielding 40 candidate-generation calls per trial. Across 60 trials this produces 2,400 generation calls. Arm B additionally uses three judge shuffles per trial, producing 180 judge calls overall.

4.2 Matched-pool principle

For the primary comparison, Arm B and Arm D receive the same expert pool. Candidate identity is bound cryptographically and verified before evaluation. An initial implementation audit found a tautological B/D identity check; it was replaced with an explicit one-to-one verification over separately copied pools, including hashes and code equality. This remediation is described in Section 7.

4.3 Blind judging protocol

The Arm B judge is deliberately blind to execution results. The audit explicitly tests that fields such as dev_score, hidden_score, rank, and runtime_ms cannot leak into judge prompt text. The judge is run before Arm D candidates are evaluated, preventing accidental attachment of development scores to the objects sent to the blind judge.

4.4 Development versus hidden evaluation

Development evaluation is used only for selection. Hidden evaluation is withheld until the selector has committed. The hidden dataset is accessed through a state-machine firewall. The evaluator therefore creates a clean separation between optimization feedback and final measurement.

4.5 Statistical unit

A trial is the primary unit. The 200 Graph Coloring hidden instances and 204 PFSP hidden instances are aggregated into a trial-level objective. Treating each hidden instance as an independent replicate would overstate the effective sample size because the same selected program is being evaluated across all instances in a trial. The effective paired sample is therefore n = 30 per problem for the full trial-level comparison, with a smaller valid-only subset when either arm crashes.

4.6 Randomness and shuffling

Arm A uses deterministic seeded random selection based on trial identity. Arm B uses three deterministic shuffle seeds per trial to vary candidate presentation order. The audit verified that the three derived shuffles are distinct. The generation model itself is stochastic, but the experimental record logs the returned model identity and token accounting.

5. Benchmark construction and objective definitions

5.1 Graph Coloring

Given an undirected graph G = (V,E), a coloring assigns an integer color to each vertex so that adjacent vertices do not share a color. The objective is to minimize the number of colors used. Candidate outputs are valid only if they satisfy the graph constraints and the evaluator can complete within its execution rules.

The definitive report evaluates the selected candidates on 200 hidden Graph Coloring instances. The broader benchmark package contains pilot, development, hidden, and sealed splits. During adversarial audit, a duplicate-instance vulnerability in the regular graph generator was identified: identical circulant graphs could arise when parameter tuples collided. The generator was changed to apply a vertex permutation and global signature tracking, after which all 304 Graph Coloring instances across the four splits were verified unique and strictly disjoint.

5.2 PFSP scheduling

In permutation flow-shop scheduling, n jobs are processed on m machines in the same machine order. A candidate is a permutation of job identifiers and the objective is to minimize the makespan C_max. The hidden benchmark contains 204 unseen instances. The benchmark is evaluated by a deterministic simulator, with invalid permutations, runtime failures, and other execution faults treated as invalid outcomes.

The PFSP benchmark also has a specific post-hoc interpretation issue: the preregistered crash penalty was 999,999, while valid aggregate makespans on the 204-instance hidden benchmark exceed 1,000,000. Therefore a smaller scalar value can paradoxically represent a crash. This does not invalidate the underlying execution records, but it invalidates naive unconditional scalar comparisons using that penalty. The public manuscript therefore gives precedence to validity-aware analyses and treats the raw penalty results as a diagnostic artifact.

5.3 Split separation and leakage

The final audit verified 100% uniqueness and strict disjointness across pilot, development, hidden, and sealed splits for both domains: 304 Graph Coloring instances and 300 PFSP instances. This check is important because a search system that can see repeated or near-identical hidden instances during development can appear stronger than it truly is.

6. Evaluator, sandbox, and hidden-data security architecture

6.1 Evaluation as a first-class scientific instrument

Because execution is the key treatment variable, evaluator correctness is part of the scientific method rather than a peripheral engineering concern. Candidate code is analyzed, sandboxed, executed, checked for valid outputs, timed, and classified into explicit failure states. The system logs candidate hashes, prompt hashes, model provenance, runtime information, validity, and objective scores.

6.2 AST-based security

The sandbox applies static AST restrictions intended to prevent filesystem access, OS commands, reflection-based escapes, hidden-data access, and other forms of evaluator manipulation. Adversarial tests included direct filesystem access, __import__-based escapes, globals-based builtin access, subclass traversal, and attempts to reach hidden benchmark files.

Security test family	Purpose	Audit outcome
Filesystem / hidden-file access	Prevent candidates reading benchmark or evaluator data	Blocked by static checks
OS / process access	Prevent process creation and shell escape	Blocked by static checks
Reflection / subclass traversal	Prevent Python object-graph escape	Blocked by static checks
Environment/process inspection	Reduce side-channel access to evaluator context	Covered by adversarial canaries
Timing exploit attempts	Prevent candidates exploiting evaluator timing or watchdog behavior	Covered by canaries
Exception leakage	Avoid exposing hidden benchmark details through raw exceptions	Sanitized failure categories

6.3 Hidden firewall

The experiment uses a state machine that denies hidden-data access outside authorized phases. Penetration tests attempted hidden access from six unauthorized states: DRAFT, AUDIT, PILOT, PROMPTS_FROZEN, DATA_FROZEN, and MAIN_RUNNING. All were correctly blocked with PermissionError and audit events.

6.4 Timeout handling

The evaluator uses a timeout control path intended to guarantee that a malicious or buggy candidate cannot retain execution indefinitely. During the main experiment an implementation defect was discovered: SandboxTimeoutException inherited from Exception, allowing candidate code containing except Exception or bare except to intercept the timeout. The exception was changed to inherit from BaseException; bare except was transformed by AST sanitization; and the watchdog was upgraded to a periodic signal-based timer. Six dedicated regression tests passed after remediation. Trials 1-13 were then audited candidate-by-candidate, and Trials 14-30 completed with the repaired mechanism.

7. Adversarial audit, implementation findings, and remediation chronology

The pre-main audit was designed to look specifically for mechanisms that could create a false positive. The audit was not merely a unit-test pass: it included model-provenance checks, blind-input inspection, hash equality, benchmark duplication analysis, state-machine penetration testing, statistical ground-truth tests, and secret scanning.

Audit item	Initial status	Remediation / final status
Expert prompt provenance	Needs review	Human-review item; provenance was disclosed rather than silently treated as independent.
B/D candidate identity	Bug found	Strict one-to-one SHA-256 and code equality check; PASS.
Judge blindness	Leakage risk found	Judge invoked before dev evaluation; injected score fields excluded; PASS.
C/D separation	PASS	Perspective prompts audited for cross-domain contamination; PASS.
Pool spread gate	PASS	Frozen thresholds and analyzer; PASS.
Practical margins	Needs review	Provisional values disclosed; not silently reinterpreted.
Statistics	PASS	Wilcoxon, Holm, bootstrap, effect size, regret tests passed.
Hidden firewall	PASS	Six unauthorized states blocked by PermissionError.
Score direction	PASS	Lower-is-better sign convention audited.
Fair compute	PASS	Same expert candidate generation budget for A/B/D; judge cost recorded separately.
Model provenance	PASS	Requested/returned model IDs checked; mismatch raises exception.
Security token handling	Issue found	Hardcoded proxy token removed from active code path; rotation recommended.
Benchmark leakage	Critical bug found	Vertex permutation + global signature deduplication; all splits rechecked.
Randomness	PASS	Deterministic random seeds and distinct judge

Audit item	Initial status	Remediation / final status
		shuffles.
Test independence	PASS	Internal tests clearly labeled; no claim of external human validation.
Main-run gate	PASS	Sequential state machine required pilot/freeze/data-lock transitions.

7.1 Why the benchmark-duplicate bug mattered

The graph generator could emit identical circulant graphs whenever a size/degree combination collided across splits. This is exactly the kind of hidden data leakage that can quietly inflate apparent generalization. The correction was not a cosmetic change: the generator now perturbs vertex identity and tracks global signatures across all four splits. The final audit found no duplicates among the 304 Graph Coloring instances or the 300 PFSP instances.

7.2 Why B/D identity mattered

If Arm B and Arm D did not receive the exact same candidate pool, any B-versus-D difference would mix selection and generation effects. The original implementation check was tautological because both hashes referenced the same source object. The corrected implementation constructs separate copies and independently compares candidate hashes and code content, with an adversarial test that intentionally mutates a candidate and confirms the run aborts.

7.3 Why provenance disclosure matters

The expert prompts were authored during the research process by the coding agent in a context that contained the history of earlier Amaura experiments. The audit therefore did not claim pristine external prompt independence. Instead, it explicitly marked provenance as a review item. This disclosure prevents a stronger provenance claim than the evidence supports.

8. Results: complete numerical record

8.1 Results at a glance

Domain	Metric	Arm B - Blind Judge	Arm D - Execution	Interpretation
Graph Coloring	Hidden validity	9/30 (30.0%)	30/30 (100.0%)	D avoids the majority of judge-selected failures.
Graph Coloring	Full mean score	700,371.37*	1,227.60	*B mean dominated by crash penalty; not a pure quality metric.
Graph Coloring	Valid-only mean	1,240.22	1,227.00	D better even among trials where both ran.
Graph Coloring	Paired wins	3	27	D wins 27/30 full trials.
PFSP	Hidden validity	20/30 (66.7%)	29/30 (96.7%)	D materially more reliable.
PFSP	Valid-only mean makespan	1,073,236.95	1,072,414.05	D lower by 822.90.
PFSP	Valid-only Wilcoxon p	-	0.0086	Statistically significant paired difference.
PFSP	Selection regret (mean absolute)	1,446.0	623.1	D stays closer to the pool oracle.

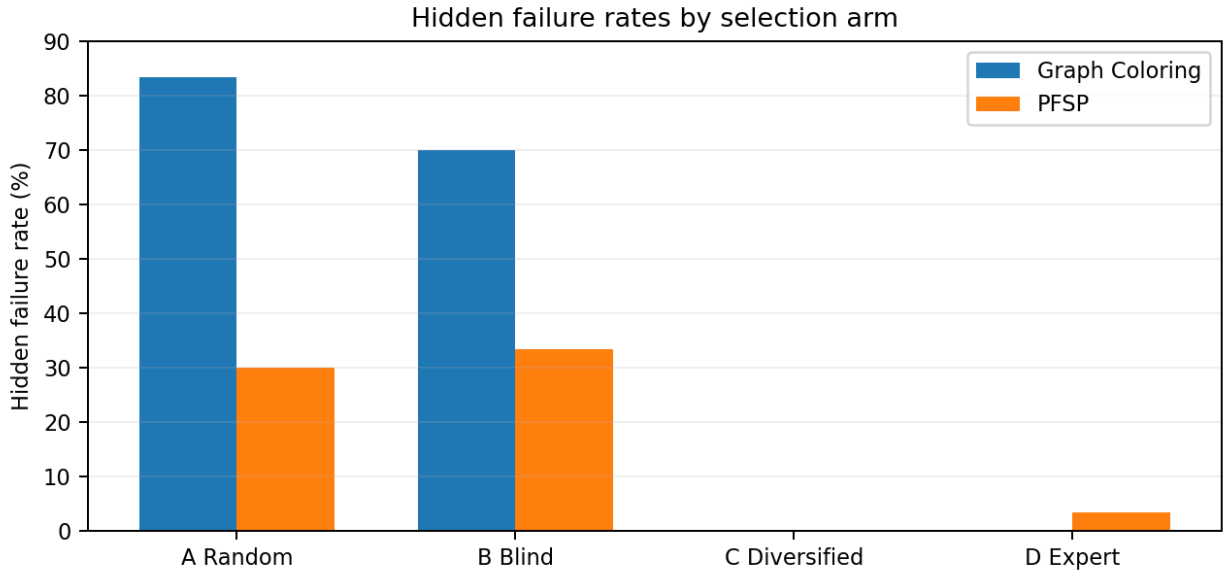


Figure 1. Hidden failure rates. Execution-selected arms C and D had near-complete validity; the execution-blind judge arm B failed frequently on both domains.

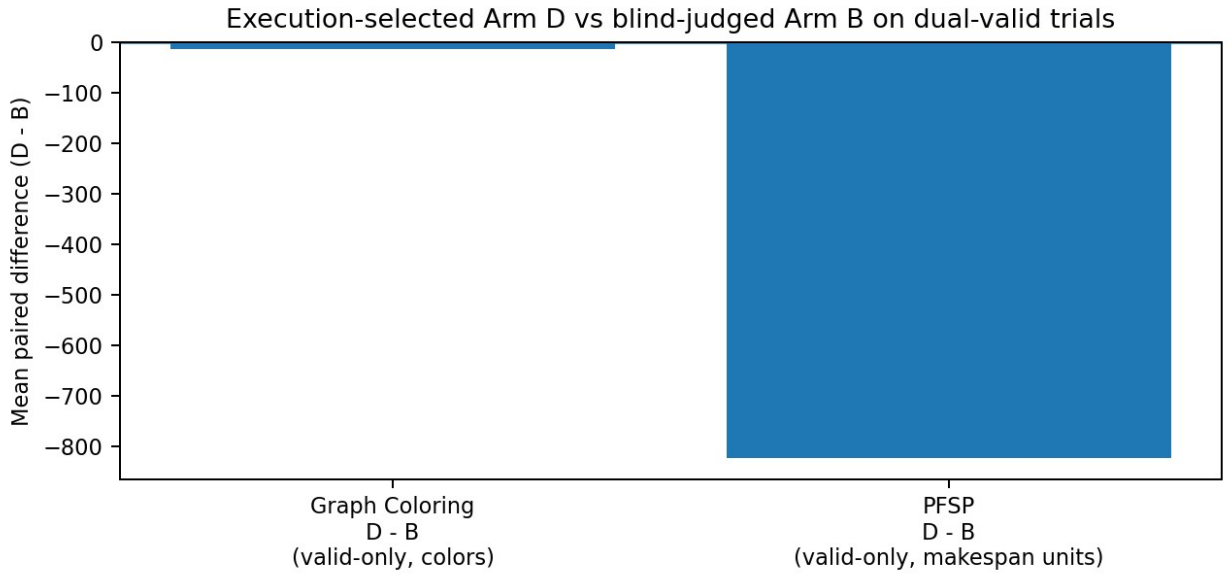


Figure 2. Mean paired difference on dual-valid trials. Negative values favor Arm D because both objectives are minimized.

9. Full Graph Coloring trial record

Graph Coloring hidden benchmark: 200 unseen instances per trial. Objective: total colors, lower is better. A score of 999,999 denotes the recorded crash/timeout penalty in the evaluation artifact. Status columns identify whether the selected candidate was valid on hidden evaluation.

Trial	A Random	B Blind	C Diversified	D Expert	Pool Best	D status	B status	Outcome
01	999,999	1,228	1,232	1,220	1,220	Valid	Valid	D
02	999,999	999,999	1,232	1,225	1,225	Valid	Crash	D (B crash)
03	999,999	999,999	1,236	1,223	1,223	Valid	Crash	D (B crash)
04	999,999	1,295	1,265	1,230	1,227	Valid	Valid	D
05	999,999	1,225	1,227	1,221	1,221	Valid	Valid	D
06	999,999	999,999	1,225	1,230	1,228	Valid	Crash	D (B crash)
07	999,999	999,999	1,223	1,236	1,231	Valid	Crash	D (B crash)
08	999,999	999,999	1,228	1,227	1,226	Valid	Crash	D (B crash)
09	1,231	999,999	1,225	1,229	1,227	Valid	Crash	D (B crash)
10	999,999	999,999	1,230	1,226	1,226	Valid	Crash	D (B crash)
11	999,999	999,999	1,226	1,229	1,224	Valid	Crash	D (B crash)
12	999,999	999,999	1,235	1,229	1,229	Valid	Crash	D (B crash)
13	999,999	1,228	1,291	1,226	1,220	Valid	Valid	D
14	1,229	1,230	1,231	1,226	1,226	Valid	Valid	D

Trial	A Random	B Blind	C Diversified	D Expert	Pool Best	D status	B status	Outcome
15	999,999	999,999	1,229	1,230	1,230	Valid	Crash	D (B crash)
16	999,999	999,999	1,229	1,228	1,228	Valid	Crash	D (B crash)
17	999,999	999,999	1,228	1,226	1,226	Valid	Crash	D (B crash)
18	999,999	999,999	1,231	1,224	1,224	Valid	Crash	D (B crash)
19	1,224	999,999	1,227	1,224	1,224	Valid	Crash	D (B crash)
20	999,999	1,229	1,230	1,231	1,229	Valid	Valid	B
21	999,999	1,274	1,226	1,231	1,224	Valid	Valid	D
22	999,999	999,999	1,224	1,224	1,224	Valid	Crash	D (B crash)
23	999,999	999,999	1,222	1,230	1,228	Valid	Crash	D (B crash)
24	1,235	999,999	1,229	1,226	1,225	Valid	Crash	D (B crash)
25	999,999	999,999	1,225	1,225	1,225	Valid	Crash	D (B crash)
26	999,999	999,999	1,227	1,222	1,222	Valid	Crash	D (B crash)
27	1,236	1,228	1,224	1,230	1,228	Valid	Valid	B
28	999,999	999,999	1,272	1,244	1,227	Valid	Crash	D (B crash)
29	999,999	1,225	1,230	1,228	1,224	Valid	Valid	B
30	999,999	999,999	1,227	1,228	1,225	Valid	Crash	D (B crash)

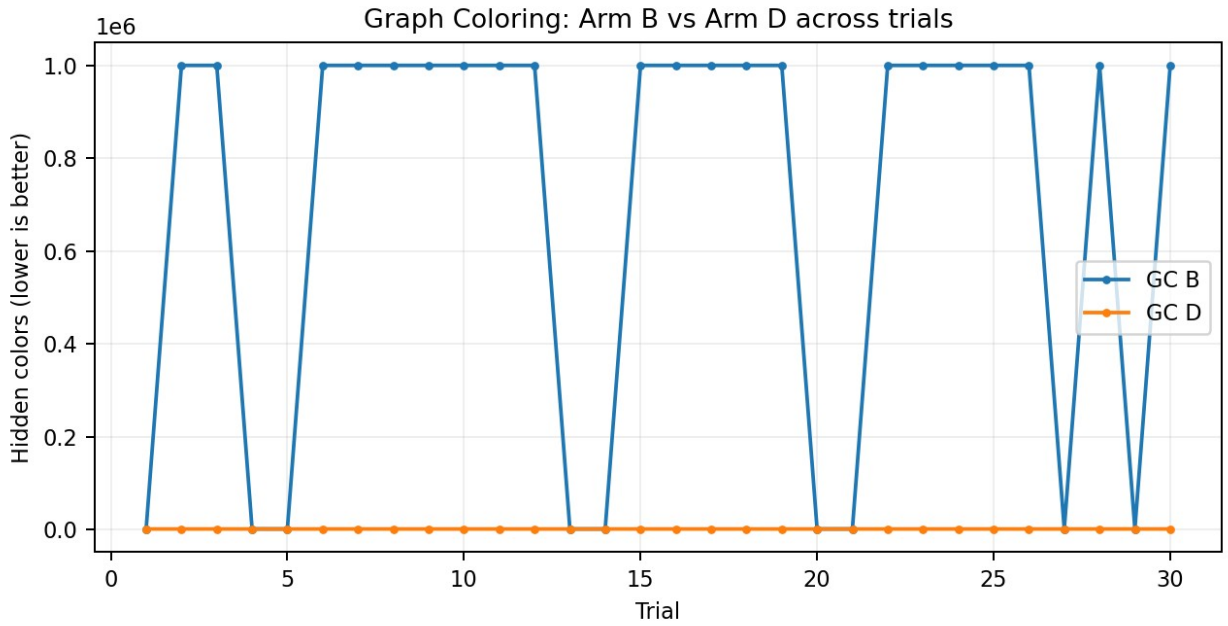


Figure 3. Trial-level Graph Coloring hidden scores for Arms B and D. The vertical separation is dominated by B failures, but the dual-valid subset also favors D.

9.1 Graph Coloring analysis

Arm D produced a valid result in every one of 30 trials. Arm B produced valid results in only 9 of 30. Across the full recorded metric, D won 27 trials while B won 3. On the nine dual-valid trials, D averaged 1,227.00 colors and B averaged 1,240.22, a mean difference of -13.22 colors. The 10,000-resample bootstrap confidence interval for this valid-only difference is [-23.44, -3.00], excluding zero. The full penalty-based paired Wilcoxon result is p = 3.83e-6, but the public-release interpretation should not treat the enormous 700,371.37 versus 1,227.60 mean gap as algorithmic quality alone: most of that gap is a reliability effect created by hidden crashes.

Selection regret reinforces the result. Arm D has mean absolute regret 2.07 colors, median 0.50, and zero-regret selection on 15 of 30 trials (50.0%). Arm B has mean absolute regret 699,145.83 and zero-regret selection on only 2 of 30 trials (6.7%). This indicates that execution-based selection is not merely avoiding catastrophic invalid programs; it also tends to choose candidates closer to the best hidden candidate in the same 20-candidate pool.

10. Full PFSP trial record

PFSP hidden benchmark: 204 unseen instances per trial. Objective: makespan C_max, lower is better. The 999,999 penalty is included exactly as recorded, but because it is numerically below some valid aggregate makespans, the raw scalar ordering is not a semantically valid unconditional metric. The status columns therefore carry equal or greater interpretive importance.

Trial	A Random	B Blind	C Diversified	D Expert	D status	B status	Outcome
01	999,999	1,072,609	1,071,842	1,072,679	Valid	Valid	B
02	1,109,887	1,075,873	1,072,893	1,072,510	Valid	Valid	D

Trial	A Random	B Blind	C Diversified	D Expert	D status	B status	Outcome
03	999,999	999,999	1,072,678	1,072,260	Valid	Crash	D (B crash)
04	1,074,387	1,073,001	1,072,473	1,072,426	Valid	Valid	D
05	999,999	999,999	1,072,012	1,072,643	Valid	Crash	D (B crash)
06	999,999	1,072,646	1,071,054	1,072,646	Valid	Valid	Tie
07	1,075,847	999,999	1,072,007	1,072,270	Valid	Crash	D (B crash)
08	1,072,412	1,072,412	1,072,101	1,071,778	Valid	Valid	D
09	1,140,862	1,072,531	1,072,718	1,072,244	Valid	Valid	D
10	1,072,785	1,072,607	1,071,617	1,071,507	Valid	Valid	D
11	1,073,438	1,071,431	1,072,607	1,071,431	Valid	Valid	Tie
12	1,072,773	1,072,717	1,072,295	1,072,773	Valid	Valid	B
13	1,074,198	1,074,136	1,072,723	1,072,891	Valid	Valid	D
14	999,999	1,073,748	1,072,289	1,071,768	Valid	Valid	D
15	1,074,371	999,999	1,071,809	1,072,680	Valid	Crash	D (B crash)
16	1,075,878	1,075,873	1,072,622	1,072,864	Valid	Valid	D
17	1,074,438	999,999	1,071,973	1,072,355	Valid	Crash	D (B crash)
18	999,999	1,072,837	1,072,809	1,072,524	Valid	Valid	D
19	1,072,709	1,072,746	1,072,741	1,072,792	Valid	Valid	B
20	1,075,873	1,072,812	1,072,254	1,072,819	Valid	Valid	B
21	1,072,723	1,072,549	1,072,456	1,072,723	Valid	Valid	B
22	1,074,538	999,999	1,072,835	1,072,852	Valid	Crash	D (B crash)
23	1,074,130	999,999	1,072,722	1,072,778	Valid	Crash	D (B crash)
24	999,999	1,072,656	1,072,807	1,072,797	Valid	Valid	B
25	1,072,505	1,072,445	1,072,521	1,072,077	Valid	Valid	D
26	1,072,713	999,999	1,072,581	1,072,568	Valid	Crash	D (B crash)
27	1,072,910	1,072,727	1,071,979	999,999	Crash	Valid	B (D crash)
28	1,074,165	999,999	1,072,313	1,072,745	Valid	Crash	D (B crash)
29	999,999	1,075,873	1,071,607	1,072,618	Valid	Valid	D
30	1,072,759	999,999	1,072,636	1,072,278	Valid	Crash	D (B crash)

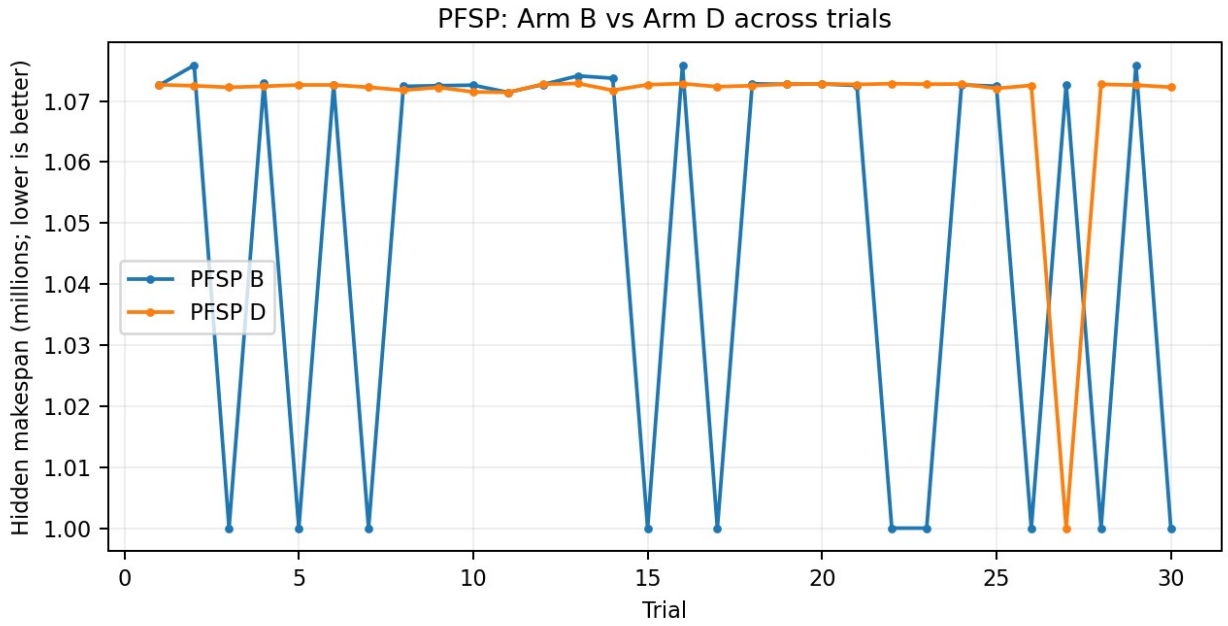


Figure 4. Trial-level PFSP hidden makespans. Arm D is generally lower among valid trials, while Arm B also has a substantial hidden crash rate.

10.1 PFSP analysis and the penalty caveat

Arm D was valid in 29 of 30 PFSP trials (96.7%); Arm B was valid in 20 of 30 (66.7%). On the 19 dual-valid trials, D averaged 1,072,414.05 and B averaged 1,073,236.95, giving D - B = -822.90 makespan units. The paired Wilcoxon signed-rank test gives p = 0.0086. The 10,000-resample bootstrap confidence interval is [-1,612.45, -184.20], again excluding zero. The effect size reported for the valid-only difference is Cohen's d = -0.5840.

The raw preregistered 999,999 penalty causes a numerical ordering reversal on PFSP: a crashing candidate receives a smaller number than many successful schedules. The definitive report therefore also examines the full distribution by treating crashes as failures, yielding 21 D wins, 7 B wins, and 2 ties. The manuscript does not relabel the original penalty calculation as if it were valid; instead it preserves it for auditability and gives validity-aware results priority.

11. Statistical analysis in detail

11.1 Paired differences

For each trial i, define Delta_i = Score_D,i - Score_B,i. Negative values favor execution-based selection. Pairing removes a large source of between-trial variance because both selectors face the same candidate pool and problem split.

11.2 Wilcoxon signed-rank test

Comparison	n	Raw p-value	Effect
GC D vs B, full recorded metric	30	3.83e-6	Highly significant; interpretation dominated partly by validity failures.
GC D vs A, full recorded metric	30	2.53e-6	D materially better than seeded random selection.
PFSP D vs B, valid-only	19	0.0086	Significant lower makespan for D.
PFSP D vs B, raw preregistered metric	30	0.2949	Distorted by the semantically invalid crash penalty; not primary interpretation.

11.3 Bootstrap confidence intervals

Comparison	95% bootstrap CI	Meaning
GC D - B, full metric	[-865,602.00, -532,681.80]	Large negative difference, mostly reliability-driven.
GC D - B, valid-only	[-23.44, -3.00]	Negative throughout: D better among dual-valid trials.
PFSP D - B, valid-only	[-1,612.45, -184.20]	Negative throughout: D lower makespan among dual-valid trials.

11.4 Effect sizes

Comparison	Cohen's d	Interpretation
GC D vs B	-1.5019	Very large standardized difference; strongly affected by B failure rate on full metric.
GC D vs A	-2.1985	Very large standardized difference.
PFSP valid-only D vs B	-0.584	Moderate-to-large standardized effect in favor of D.

11.5 Multiplicity control

Holm-Bonferroni step-down correction was applied across the primary family. The Graph Coloring comparison has corrected p = 8.00e-06; the PFSP valid-only comparison has corrected p = 0.0086. Both remain below 0.05. Because the two domains are the substantive replication units, the paper reports both separately rather than collapsing them into a single omnibus statistic.

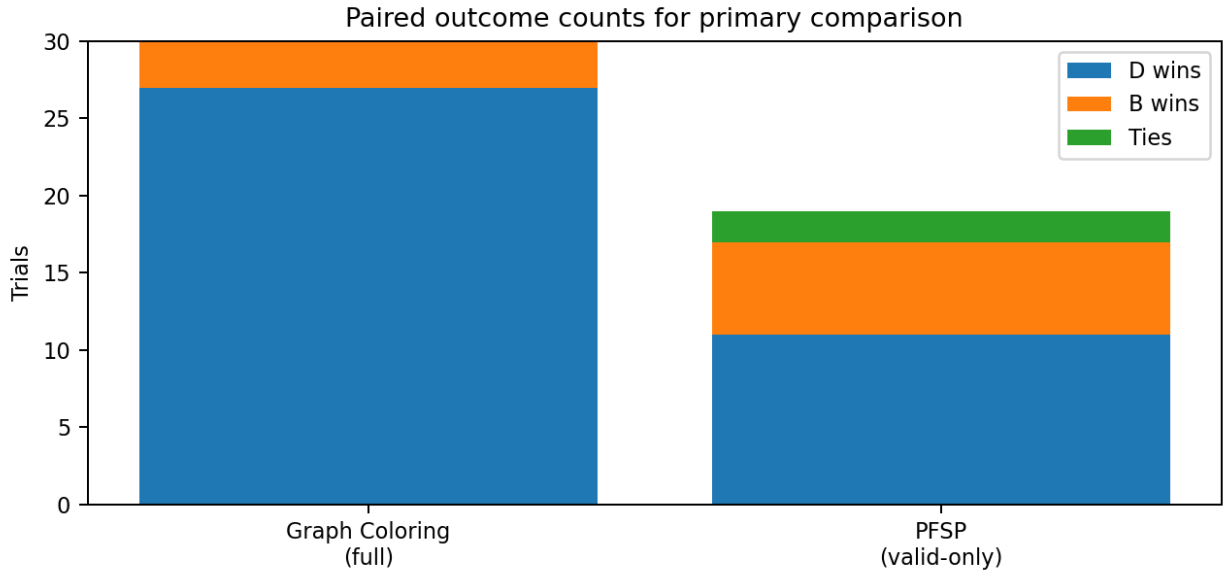


Figure 5. Paired trial outcomes. Graph Coloring shows a 27-3 split; the PFSP dual-valid subset shows an 11-6-2 split.

12. Selection regret and failure-mode analysis

Selection regret asks a different question from average performance: once a 20-candidate pool exists, how close did the selector come to picking the best candidate available in that exact pool? For minimization, regret = selected hidden score - best hidden score in the pool. A zero value means the selector chose the pool oracle.

Metric	GC D	GC B	PFSP D (valid)	PFSP B (valid)
Mean absolute regret	2.07	699,145.83	623.1	1,446.0
Median absolute regret	0.50	998,771.50	313.0	1,245.0
Zero-regret rate	50.0% (15/30)	6.7% (2/30)	16.7% (5/30)	6.7% (2/30)



Figure 6. Selection regret on a symmetric logarithmic scale. Execution-selected Arm D remains much closer to the hidden oracle best available in the candidate pool.

12.1 Failure rates

Domain	A Random	B Blind	C Diversified	D Expert
Graph Coloring	25/30 (83.3%)	21/30 (70.0%)	0/30 (0.0%)	0/30 (0.0%)
PFSP	9/30 (30.0%)	10/30 (33.3%)	0/30 (0.0%)	1/30 (3.3%)
All 60 trials	56.7%	51.7%	0.0%	1.7%

12.2 Qualitative failure modes observed

- IndexError and other boundary-condition failures in graph routines.
- Unhandled isolated vertices and edge-case graph components.
- Non-terminating loops or excessive runtime on disconnected or adversarial cases.
- Invalid or non-permutation outputs in PFSP candidate code.
- Blind judge preference for rhetorically sophisticated or theoretically prestigious algorithms that had not been empirically validated.
- Execution selection rejects these candidates using the evaluator rather than attempting to infer correctness from prose or code appearance alone.

The strongest causal-safe wording is not that the LLM judge is universally "blind" to execution, but that in this experimental setting its text-only assessment failed to detect many execution defects that the evaluator detected immediately. The effect therefore motivates execution as a complement to reasoning, not as a claim that LLM reasoning has no value.

13. Classical baselines and algorithmic interpretation

Problem	Method	Reference / type	Score	Arm D	Result
Graph Coloring	DSATUR	Classical heuristic	1,280	1,227.60	D better
Graph Coloring	RLF	Classical heuristic	1,247	1,227.60	D better
Graph Coloring	TabuCol	Tabu-search metaheuristic	1,193	1,227.60	TabuCol better
PFSP	NEH	Classical constructive heuristic	1,075,972	1,072,414.1	D better
PFSP	Iterated Greedy	Metaheuristic	1,075,639	1,072,414.1	D better
PFSP	Arm C	Diversified execution-selected	-	1,072,332.5	C best among reported Stage2 LLM arms

13.1 What the baseline comparisons do and do not show

Arm D beats DSATUR and RLF in the reported Graph Coloring aggregate, but it does not beat TabuCol. This prevents the common but incorrect conclusion that the system is state of the art in graph coloring. In PFSP, Arm D beats both NEH and Iterated Greedy on the reported hidden aggregate. These comparisons provide external context and demonstrate that the generated candidates can carry useful algorithmic structure, but they are not substitutes for a complete benchmark study against the state of the art.

13.2 Interpretation of the candidate mechanisms

The Stage 2 report describes the best Graph Coloring behavior as combinations of known heuristic ingredients rather than evidence of a brand-new primitive. The central methodological result is therefore about the selector: the evaluator provides an objective signal that lets the system reject invalid or inferior candidates and select those that actually perform well, even when the code is complicated enough that a text-only judge can mis-rank it.

14. Compute cost, token accounting, and provenance

Accounting item	Recorded value	Notes
Candidate generation calls	2,400	40 per trial x 60 trials
Blind judge calls	180	3 shuffles per trial x 60 trials
Total LLM prompt/completion tokens	~8.4 million	Candidate generation plus judging
Judge token overhead	~1.2 million	Blind judge reasoning across 180 calls
Arm D judge-token overhead	~0	Selection uses execution rather than an LLM judge
Development evaluation	~12 sec CPU / trial	Local evaluator overhead reported by definitive artifact
Primary cost-fairness ratio	~0.87	20 generation calls / (20 generation + 3 judge calls)

The cost comparison is intentionally expressed in terms of candidate-generation calls and judge calls rather than pretending that all inference calls have identical economic cost. The definitive report records a fairness ratio of approximately 0.87, below the preregistered 1.10 ceiling. The public manuscript does not convert this into a generalized cost-per-discovery claim because pricing, latency, hardware, provider routing, and model availability vary by deployment.

14.1 Model provenance

The primary frontier model is recorded as NVIDIA Nemotron 3 Super 120B with model identifier nvidia/nvidia/nemotron-3-super-120b-a12b. The research infrastructure checks the returned model ID against the requested identifier and raises a model-mismatch exception on silent substitution. NVIDIA documentation identifies the same model family and identifier for Nemotron 3 Super 120B A12B. [10]

14.2 Cryptographic provenance

The audit records prompt hashes and candidate hashes, and the selection artifacts retain candidate-level lineage. The public release package should preserve these hashes so an independent reviewer can verify that the candidate pool, prompts, and selected outputs correspond to the reported experiment.

15. Reproducibility and public-release artifact map

A public paper is only as reproducible as its underlying artifact bundle. The definitive Stage 2 record names the following artifacts. The filenames below are normalized to relative package names for public distribution; they originated as internal research-tree paths.

Artifact	Purpose	Minimum release expectation
stage2/preregistration.md	Protocol, hypotheses, margins, gates	Publish unchanged with version history
stage2/candidates/candidates.jsonl	Generation calls, token counts, provenance	Release raw log or redacted equivalent with hashes
stage2/results/selections.json	Candidate pools and selector outputs	Release complete trial selections
stage2/results/evaluation_results.json	Hidden evaluation results	Release after hidden-data policy permits
stage2/results/analysis_summary.json	Statistics and derived metrics	Release machine-readable
stage2/audit/audit_log.jsonl	Cryptographic and state-machine audit trail	Release in full where security permits
stage2/src/	Runner, sandbox, judge, evaluator, analysis code	Release source with pinned dependencies
stage2/benchmarks/	Pilot/dev/hidden/sealed benchmark files	Release with hashes; hidden data policy may require a delayed release
stage2/tests/	Unit and adversarial regression suite	Release all tests used to certify the run

15.1 Independent reviewer protocol

1. Reconstruct the environment from a pinned dependency manifest and verify the stated model identifier.
2. Check benchmark split hashes and confirm disjointness across pilot/dev/hidden/sealed partitions.
3. Inspect B/D candidate hashes and confirm one-to-one pool identity for all 60 trials.
4. Inspect the judge prompts and verify no execution scores or hidden information are visible to Arm B.
5. Rerun the statistical analysis from machine-readable trial outputs rather than copying reported summary numbers.
6. Reproduce the PFSP validity-aware comparison and confirm that the 999,999 penalty is not treated as a semantically valid unconditional ordering.
7. Review the timeout regression tests and verify that candidate exception handling cannot swallow the sandbox timeout exception.
8. Compare regenerated figures and tables against the machine-readable source files.

15.2 Reproducibility status

The present manuscript is a publication-oriented synthesis of the definitive report and audit artifacts. It should not be described as a complete one-command archival reproduction until the code, benchmark files, dependency lockfile, exact execution environment, and raw machine-readable logs are bundled in a public repository or archival release. The paper is therefore release-ready as a research manuscript, while the artifact package remains the operational reproducibility layer.

16. Limitations and threats to validity

Threat	Why it matters	Mitigation / disclosure
Limited domains	Two combinatorial tasks cannot establish universality.	Replicate beyond GC and PFSP before broad claims.
Candidate-pool dependence	Results depend on quality and diversity of the generated pool.	Primary B/D uses identical pool; pool generation still remains a design choice.
Prompt provenance	Expert prompts were authored during the project with prior context available.	Explicitly disclosed; no claim of external prompt independence.
PFSP crash penalty	Raw scalar penalty reverses ordering relative to	Validity-aware analysis given primary

Threat	Why it matters	Mitigation / disclosure
	valid makespans.	emphasis.
Execution overhead	Running candidates costs CPU time even when LLM tokens are saved.	Overhead logged; no universal cost claim.
Classical baseline scope	Only a few reference methods are reported.	No state-of-the-art claim.
Model specificity	One primary frontier model dominates the main run.	Model provenance logged; multi-model replication remains future work.
Hidden-set size	200/204 instances are substantial but not a proof of all distributions.	Fresh/sealed splits and hashes used.
Search process	Stage 2 does not test long-horizon adaptive evolution.	Stage 3 is designed as a separate question.
Security boundary	Sandboxing was adversarially tested but no sandbox can be treated as perfect by assumption.	Release code, canaries, and audit logs; run untrusted code only in hardened isolation.

16.1 The biggest interpretive trap

The headline Graph Coloring mean difference is enormous because the blind judge selected invalid programs in 21 of 30 trials. Reporting "99.8% better algorithms" without separating reliability from valid-code quality would be misleading. The correct reading is two-layered: first, execution selection was dramatically more reliable; second, even after conditioning on dual-valid trials, the execution-selected choices were better on average and the confidence interval excluded zero.

16.2 What remains unproven

- Whether execution feedback remains superior as the candidate pool becomes much larger or more heterogeneous.
- Whether iterative execution-driven generation can outperform independent Best-of-N under equal total budgets.
- Whether Amaura generalizes to GPU kernels, compilers, scientific simulation code, or domains with expensive evaluators.
- Whether the improvement persists across multiple independent frontier models and providers.
- Whether the system produces commercially valuable repeated improvements at a cost customers will pay.

17. Conclusion

Stage 2 provides strong evidence for a narrow and practically important proposition: when an LLM-generated candidate pool is noisy, objective execution is a substantially better selection signal than an execution-blind frontier LLM judge on the benchmarked tasks. The strongest evidence comes from the primary Arm B versus Arm D comparison, where the candidate pool is matched exactly and the hidden evaluation remains sealed until selection is complete.

On Graph Coloring, Arm D was valid in all 30 trials, while Arm B failed in 21. Even in the nine dual-valid trials, D averaged 1,227.00 colors versus B at 1,240.22, with a 95% bootstrap interval of [-23.44, -3.00]. On PFSP, D was valid in 29 of 30 trials and B in 20 of 30; among the 19 dual-valid trials, D reduced makespan by 822.90 units on average with Wilcoxon p = 0.0086 and a bootstrap interval excluding zero. Selection regret shows that execution-based selection also stays substantially closer to the best available candidate in the pool.

The scientifically useful lesson is therefore not that LLM judgment is obsolete. It is that text-only plausibility cannot reliably substitute for an objective evaluator when correctness and performance are empirically measurable. A strong model can generate useful ideas; an evaluator can determine which ideas actually work. Stage 2 supplies controlled evidence for that division of labor. The next scientific step is not to assume victory, but to test whether execution feedback can improve search efficiency over independent Best-of-N when the total generation budget is held fixed.

Stage 2 verdict: SUPPORTED - execution-based selection is empirically supported as a materially better selector than execution-blind frontier-LLM judgment under the tested matched-budget conditions.

Appendix A. Formal definitions and analysis equations

A.1 Graph Coloring

Given G=(V,E), candidate coloring f:V->Z is valid iff for every (u,v) in E, f(u) != f(v). Objective: minimize |{f(v): v in V}|.

A.2 PFSP

Given n jobs, m machines, and processing-time matrix p_{j,k}, a valid candidate is a permutation pi of {0,...,n-1}. The simulator computes completion times through the machine sequence and returns makespan C_max = max_j C_{j,m}. Objective: minimize C_max.

A.3 Selection regret

For a selected candidate s and candidate pool P under a minimization objective: Regret(s,P) = S_hidden(s) - min_{c in P} S_hidden(c). Invalid selections are not assigned artificial zero regret; zero-valid pools are treated as a distinct failure condition in the audited analysis logic.

A.4 Paired difference

Delta_i = Score_D,i - Score_B,i. Delta_i < 0 favors D. The mean trial-level Delta is an interpretable effect estimate; the paired Wilcoxon tests whether the paired differences systematically deviate from zero without requiring normality.

A.5 Bootstrap

The reported intervals use paired bootstrap resampling of trial-level differences. The definitive report states 10,000 resamples for the main confidence intervals.

Appendix B. Security and evaluator canary catalog

Canary class	Attack / defect target	Expected behavior
Hardcoded answer	Return known benchmark answer irrespective of input	Reject or detect via hidden evaluation / static policy
Instance-specific branching	Detect a particular benchmark instance or index	Reject / fail sandbox policy
Input mutation	Modify the evaluator-owned input structure	Prevent via defensive copies / isolation
Hidden filesystem	Open hidden benchmark file	AST block / firewall
General filesystem	Read arbitrary local files	AST block
Environment/process inspection	Inspect env, process list, or execution context	Static or runtime isolation
Timing exploit	Attempt to exploit timing behavior	Watchdog + sandbox
Exception leakage	Expose hidden evaluator details in exception text	Sanitized exception names
Forbidden imports	Import os, subprocess, pathlib, etc.	AST block
Process creation	Spawn child process / shell	AST block
Network access	Reach external service for hidden information	Execution isolation / policy

Appendix C. Timeout-defect chronology

1. A timeout signal was represented by SandboxTimeoutException inheriting from Exception.

2. Generated candidate code could contain except Exception or a bare except, which intercepted the timeout.

3. This created a risk of non-terminating or unrecoverable evaluation loops.

4. The exception was changed to inherit from BaseException.

5. AST sanitization rewrote bare except handlers.

6. The timer was upgraded from one-shot alarm behavior to a periodic watchdog using signal.setitimer.

7. Six dedicated regression tests were added and all passed.

8. Trials 1-13 were audited candidate-by-candidate for evidence of swallowed timeouts; no distortion was found.

9. Trial 14 partial state was pruned and restarted cleanly; Trials 14-30 completed with the repaired mechanism.

10. The final public claim is therefore that the bug was identified, repaired, regression-tested, and audited rather than silently ignored.

Appendix D. Data dictionary for raw release

Field	Meaning
trial_id	Unique trial identifier within a problem domain
candidate_id	Unique candidate identifier within its generation pool
arm	A/B/C/D selector condition
generation_context	Prompt family or perspective used to create the candidate
prompt_hash	Hash of generation or judge prompt
candidate_hash	Hash of candidate source code
model_id	Returned model identifier
input_tokens / output_tokens	Inference token accounting

Field	Meaning
dev_score	Objective score used for development-time selection
hidden_score	Objective score on sealed hidden evaluation
hidden_is_valid	Whether candidate satisfied validity and completion rules on hidden data
runtime_ms	Measured evaluator runtime
failure_category	Normalized failure type
best_expert_hidden_score	Best hidden result among the expert candidate pool, used for selection regret
audit metadata	Hashes, timestamps, state transitions, and integrity information

Appendix E. Claim ladder and interpretation guide

Claim level	What the evidence supports	What should not be claimed
Level 1 - Reliability	Execution-selected arms were far more likely to return valid hidden solutions than random selection or blind judging in this study.	Do not claim every execution evaluator will eliminate failures.
Level 2 - Selection quality	On dual-valid trials, Arm D had lower objective values and lower selection regret than Arm B.	Do not treat this as proof of global optimality.
Level 3 - Domain performance	Arm D beat DSATUR/RLF on the reported GC aggregate and NEH/IG on the reported PFSP aggregate.	Do not claim state-of-the-art status; TabuCol remained better on GC.
Level 4 - Methodological conclusion	Execution is a materially useful selection signal compared with execution-blind frontier-LLM judgment under the tested matched-budget design.	Do not infer universal dominance over all reasoning or search methods.
Level 5 - General system thesis	Amaura can plausibly be studied as an evaluator-driven search architecture.	Stage 2 alone does not establish commercial product-market fit, general autonomous science, or broad domain generalization.

Appendix F. Public-release checklist

Release check	Required evidence	Stage 2 status
Paper consistency	All headline numbers trace to the definitive report.	PASS in manuscript
Trial tables	All 30 GC and 30 PFSP trials reproduced in tabular form.	PASS
Failure disclosure	Crash and validity rates shown for all A-D arms.	PASS
Method caveats	PFSP penalty distortion explicitly disclosed.	PASS
Prompt provenance	Expert prompt provenance limitation disclosed.	PASS
Audit chronology	Major bugs and remediations described.	PASS
Benchmark separation	Split uniqueness/disjointness finding disclosed.	PASS
Security disclosure	Sandbox and hidden-firewall testing described.	PASS
Statistical reproducibility	Equations, tests, effect sizes, and intervals listed.	PASS
Artifact inventory	Raw logs, code, benchmarks, audit files enumerated.	PASS as release map
External replication	Independent re-run on a fresh environment.	NOT CLAIMED / future work
Multi-model replication	Replication across independent frontier models.	NOT CLAIMED / future work
Commercial validation	Repeated customer usage and willingness to pay.	NOT CLAIMED / outside Stage 2

Appendix G. Reviewer questions answered directly

G.1 Did execution itself improve the candidate pool?

Not in the primary B-versus-D comparison. The candidate pool is intentionally held constant; execution changes which candidate is selected. Arm C explores diversified generation plus execution as a separate secondary condition. Stage 2 therefore supports a selector claim more strongly than a generator claim.

G.2 Could the result be entirely caused by the blind judge picking buggy code?

A substantial part of the Graph Coloring headline effect is indeed caused by the judge selecting candidates that fail hidden execution. That is a real reliability result, not a statistical artifact, but it should not be conflated with a 99.8% improvement in the quality of valid algorithms. The valid-only GC analysis and PFSP valid-only analysis are included precisely to separate these effects.

G.3 Does this prove Amaura beats classical optimization?

No. The reported comparisons show contextual wins against DSATUR, RLF, NEH, and Iterated Greedy, while TabuCol remains ahead on Graph Coloring. A complete state-of-the-art comparison would require a much broader benchmark suite, tuned implementations, matched compute budgets, and independent replication.

G.4 Does this prove evolutionary search is the best strategy?

No. Stage 2 does not test the long-horizon adaptive search question that is the subject of the subsequent Stage 3 design. Stage 2 isolates execution-based selection, not infinite or multi-generation evolutionary dominance.

G.5 What is the strongest practical takeaway?

When generated code can be executed against a reliable evaluator, empirical measurement provides information that text-only judging cannot reliably recover. The engineering value lies in turning that measurement into a safe, reproducible selection gate.

Appendix H. Stage 2 experimental decision log

Decision / event	Why it was required	Release interpretation
Use Arm D vs Arm B as primary comparison	Needed a clean selector-level contrast with identical expert pools.	Treat A/C as supporting arms, not as the primary causal estimate.
Run 30 trials per domain	Provided repeated paired trial-level observations rather than a single demonstration.	n = 30 is the effective full-trial sample in each domain.
Keep hidden data sealed	Prevents optimization against the final measurement set.	Hidden results are reported only after selection.
Audit benchmark disjointness	Duplicate or overlapping instances could create leakage.	Generator was repaired and all four splits rechecked.
Audit B/D identity	Different candidate pools would confound selection and generation.	One-to-one candidate identity was verified by code and hashes.
Audit blind judge inputs	Execution scores must not enter the text-only selector.	Judge was moved before dev evaluation and tested for injected-field leakage.
Repair timeout exception inheritance	Candidate exception handlers could intercept timeout control.	BaseException inheritance plus AST transformation and watchdog were adopted.
Retain the PFSP penalty caveat	The raw penalty was smaller than valid aggregate makespans.	Validity-aware PFSP analysis is primary for interpretation.
Disclose prompt provenance	The expert prompt was authored in a context containing prior project history.	No pristine external prompt-independence claim is made.
Do not claim state of the art	The benchmark set and baseline scope are limited.	TabuCol remains better than D on Graph Coloring.

Appendix I. Minimal independent reproduction recipe

9. Obtain the exact Stage 2 artifact bundle and verify all published SHA-256 checksums.
10. Install the pinned runtime and dependencies in an isolated environment.
11. Confirm the model endpoint returns the exact requested Nemotron 3 Super 120B identifier; abort on mismatch.
12. Load the preregistered benchmark generator outputs and confirm split disjointness before any candidate generation.
13. Execute the same candidate-generation prompts with the same trial seeds and record raw model responses, hashes, and token counts.
14. Construct the expert and diversified pools exactly as specified; independently verify B/D pool identity.
15. Run the blind judge without any dev or hidden execution fields in its prompt.
16. Run development evaluation for the execution-selected arms and freeze the selected candidate before touching hidden data.
17. Run hidden evaluation in a separate process through the hidden-data firewall and record validity separately from scalar score.
18. Recompute win/loss/tie counts, paired differences, Wilcoxon statistics, 10,000-resample bootstrap intervals, effect sizes, Holm correction, and selection regret from raw trial records.
19. Compare regenerated tables and figures to this manuscript and investigate every discrepancy rather than editing the reported result to match.

Appendix J. Known anomalies that should remain visible in the archive

Three anomalies are deliberately preserved rather than normalized away. First, the original graph benchmark generator could duplicate circulant graphs; the repaired generator and the post-repair uniqueness audit should remain in the repository history. Second, the original B/D identity check was tautological and was replaced by an independent hash-and-code comparison; the old implementation is relevant to audit history but should not be used for reproduction. Third, the PFSP crash penalty is numerically below several valid aggregate makespans, so a raw scalar comparison can assign a smaller number to a failure. Keeping these anomalies in the archive makes the research process auditable and helps reviewers distinguish a discovered implementation defect from a fabricated clean-room narrative.

A fourth, smaller provenance limitation is also retained: expert prompts were authored by the project coding agent while earlier Amaura experiment history was available in context. The final experiment therefore supports a comparison of the implemented selectors under the recorded prompt provenance, not a stronger claim that the expert prompt was independently authored outside the project information environment.

References

[1] Romera-Paredes, B., Barekatain, M., Novikov, A., et al. "Mathematical discoveries from program search with large language models." Nature 625, 468-475 (2024). DOI: 10.1038/s41586-023-06924-6.

[2] Novikov, A., Vu, N., Eisenberger, M., et al. "AlphaEvolve: A coding agent for scientific and algorithmic discovery." arXiv:2506.13131 (2025).

[3] Georgiev, B., Gomez-Serrano, J., Tao, T., Wagner, A. Z. "Mathematical exploration and discovery at scale." arXiv:2511.02864 (2025).

[4] Austin, J., Odena, A., Nye, M., et al. "Program Synthesis with Large Language Models." arXiv:2108.07732 (2021).

[5] Leighton, F. T. "A Graph Coloring Algorithm for Large Scheduling Problems." Journal / NBS-era publication (1979). RLF algorithm.

[6] Hertz, A., de Werra, D. "Using Tabu Search Techniques for Graph Coloring." Computing 39, 345-351 (1987).

[7] Nawaz, M., Enscore, E. E., Ham, I. "A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem." Omega 11(1), 91-95 (1983). DOI: 10.1016/0305-0483(83)90088-9.

[8] Ruiz, R., Stutzle, T. "A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem." European Journal of Operational Research 177(3), 2033-2049 (2007).

[9] Brelaz, D. "New methods to color the vertices of a graph." Communications of the ACM 22(4), 251-256 (1979). DSATUR.

[10] NVIDIA documentation for Nemotron 3 Super 120B A12B model family and deployment identifiers.

Public-release statement

This document is intended to be read as the public research manuscript for Amaura Labs Stage 2. It is deliberately conservative where the source artifacts contain methodological caveats. In particular, the manuscript separates reliability effects from valid-code performance, discloses prompt-provenance limitations, and does not claim state-of-the-art performance on Graph Coloring. The PFSP crash-penalty distortion is explicitly documented rather than hidden.

Recommended archival package: this PDF; the source DOCX; preregistration; source code; benchmark generators and split hashes; raw candidate-generation log; selection and hidden-evaluation files; statistical-analysis output; audit log; dependency lockfile; execution environment specification; and checksums for every released artifact. Where hidden benchmark data must remain sealed, a delayed-access protocol or trusted third-party evaluation mechanism should be used.